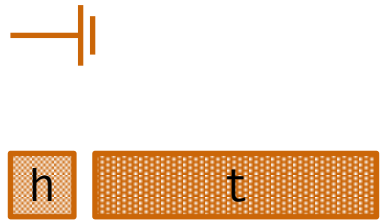
Lecture 16
Synthesis from Refinement Types

With slides from Nadia Polikarpova

Refinement types

```
data SList e where
  Nil :: SList e
  Cons :: h:e →
          t:SList {v:e | v ≥ h} →
          SList e
```

sorted lists




```
measure elems :: SList a -> Set a where
  Nil -> []
  Cons x xs -> elems xs + [x]
```

Refinement types as specs

[Rondon et al. PLDI'08]

```
// Insert x into a sorted list xs
insert :: x:e  →  xs:SList e  →
          {v:SList e | elems v = elems xs ∪ {x}}
insert x xs =
  match xs with
    Nil → Nil
    Cons h t →
      if x ≤ h
        then Cons x xs
        else Cons h (insert x t)
```

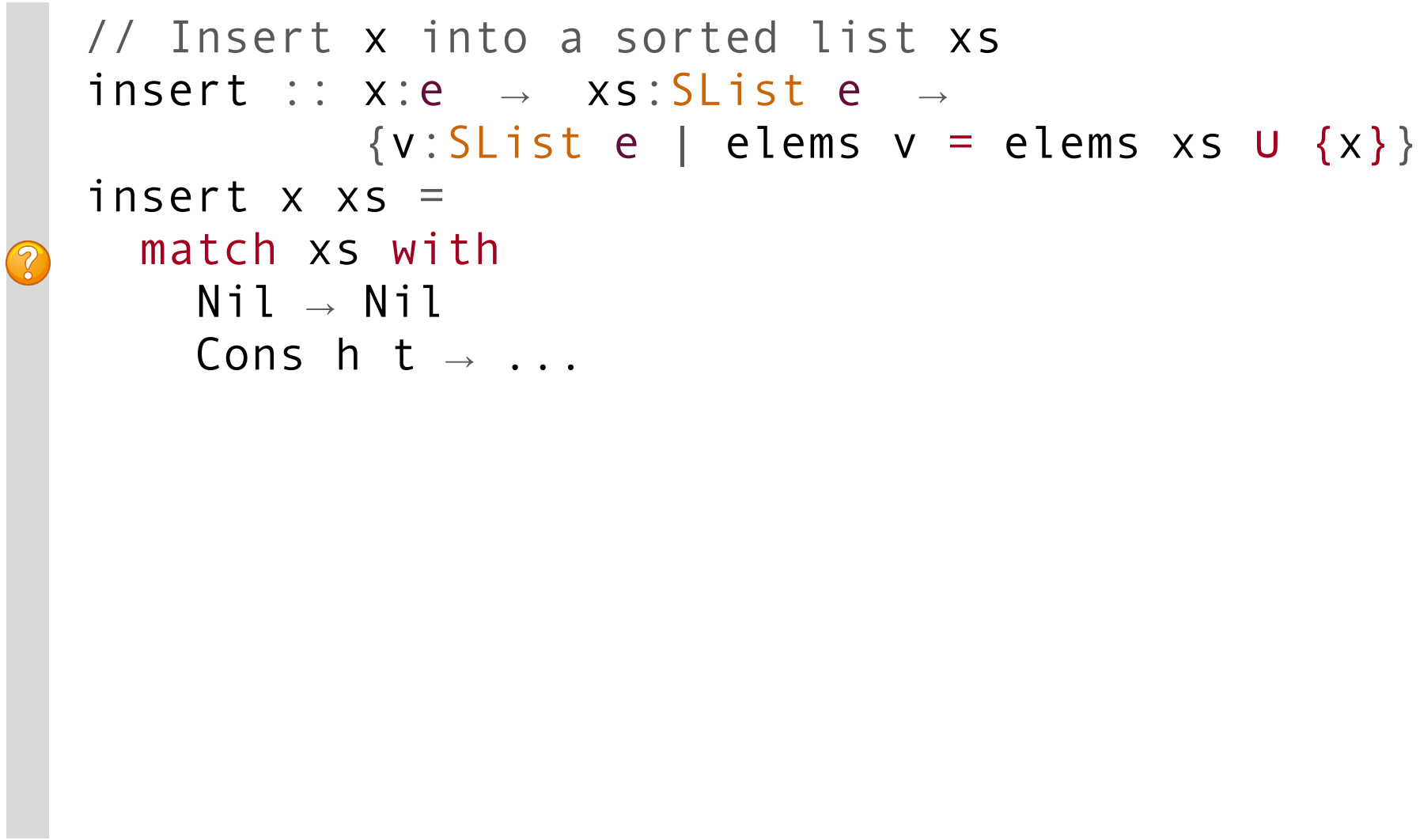
Expected

{v:SList e|elems v = elems xs ∪ {x}}

and got

{v:SList e|elems xs ⊆ elems v}

Incomplete programs?



```
// Insert x into a sorted list xs
insert :: x:e  →  xs:SList e  →
          {v:SList e | elems v = elems xs ∪ {x}}
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

Bidirectional type checking

```
x:e  →  xs:SList e  →
  {v:SList e | elems v = elems xs ∪ {x}}
```



```
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

Bidirectional type checking

```
{v:SList e | elems v = elems xs ∪ {x}}
```



```
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

Bidirectional type checking

```
{v:SList e | elems v = {x}}
```



```
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

Bidirectional type checking

```
{v:SList e | elems v = {x}}
```



```
insert x xs =
  match xs with
    Nil → Nil
    Cons h t → ...
```

Round-trip type checking

```
x:e  →  xs:SList e  →
  {v:SList e | elems v = elems xs ∪ {x}}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```

Round-trip type checking

```
SList e
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```

Round-trip type checking

```
SList e
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```



Round-trip type checking

```
SList {v:e | v ≥ h}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```



Round-trip type checking

```
{v:e | v ≥ h}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```



Round-trip type checking

```
{v:e | v ≥ h}
```



```
insert x xs =
  match xs with
    Nil → Cons x Nil
    Cons h t →
      Cons h (insert x ...)
```



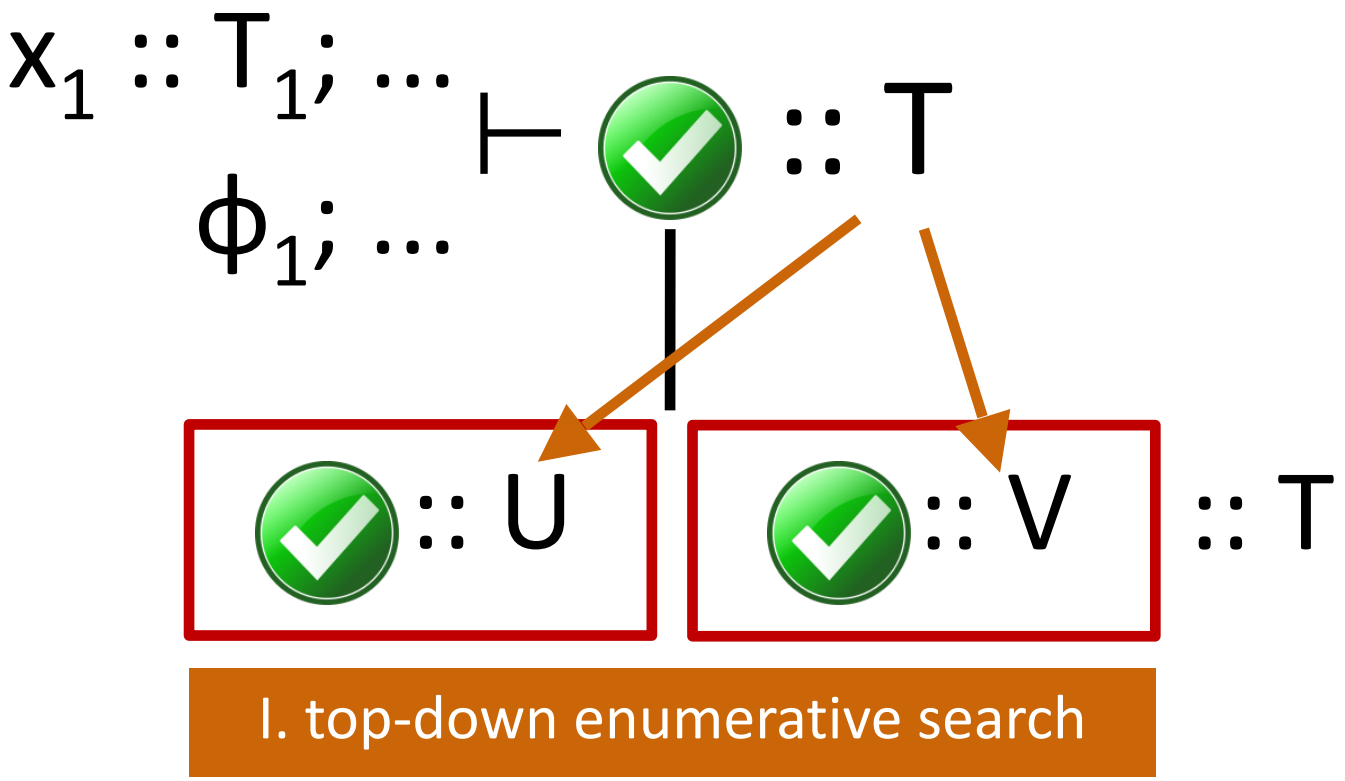


Type-driven Synthesis

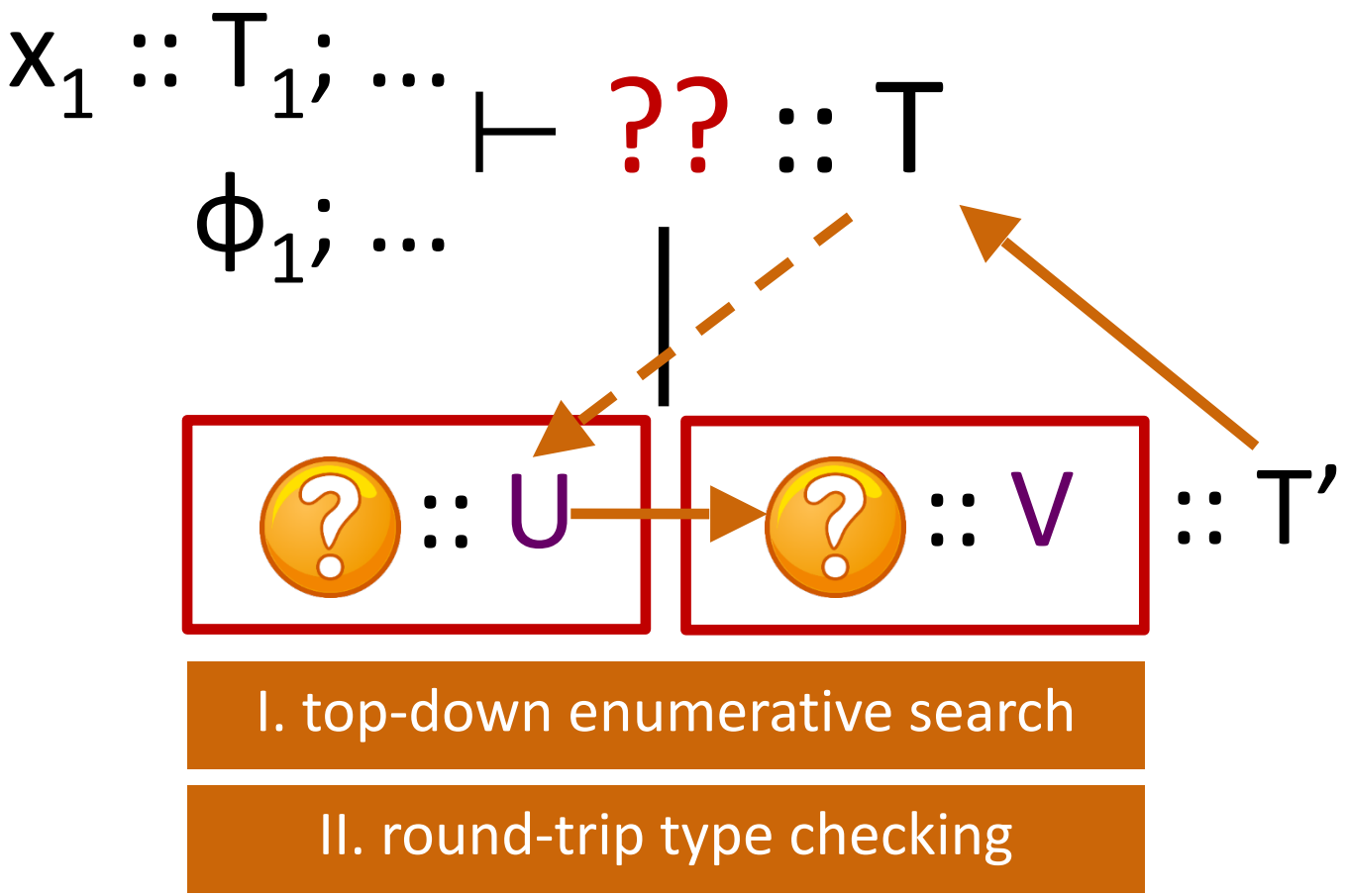


http://tiny.cc/synquid

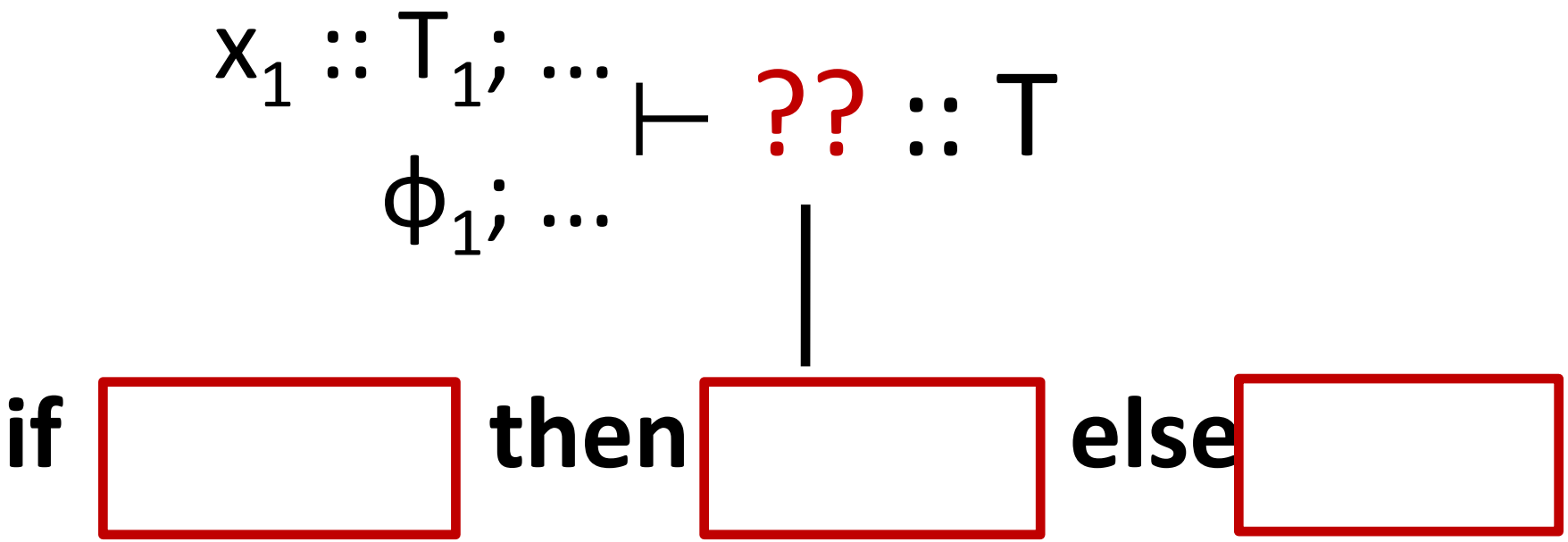
Synthesis from refinement types



Synthesis from refinement types



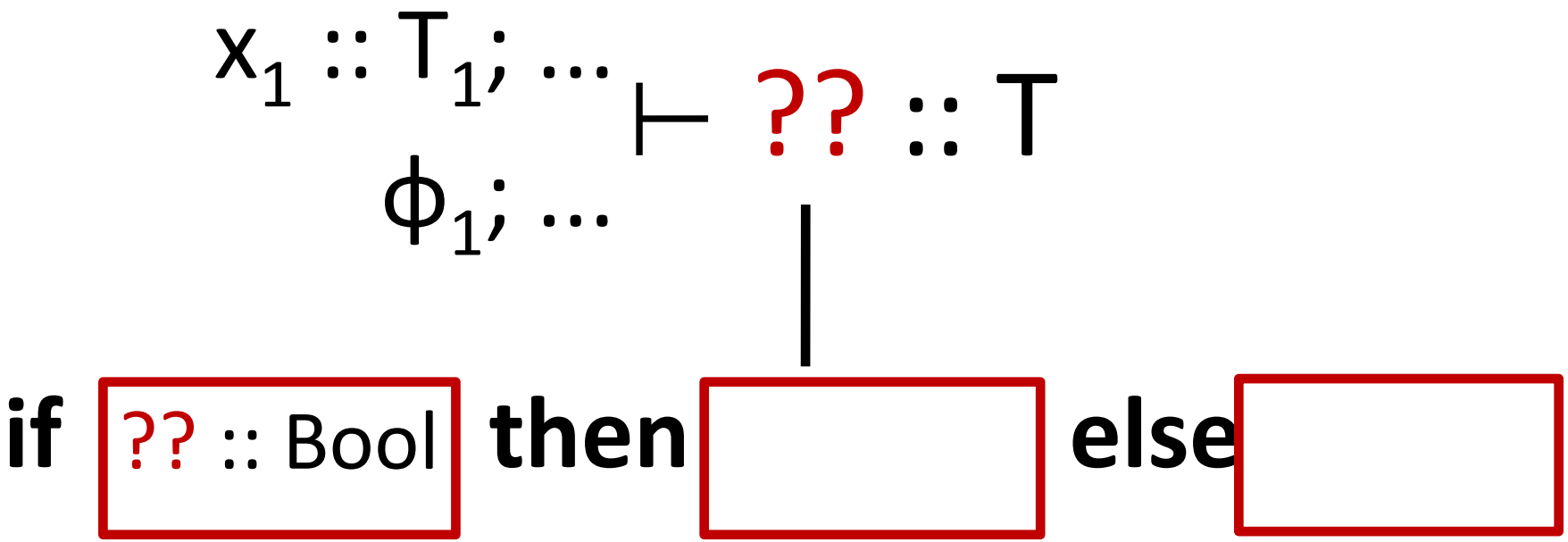
Synthesis from refinement types



I. top-down enumerative search

II. round-trip type checking

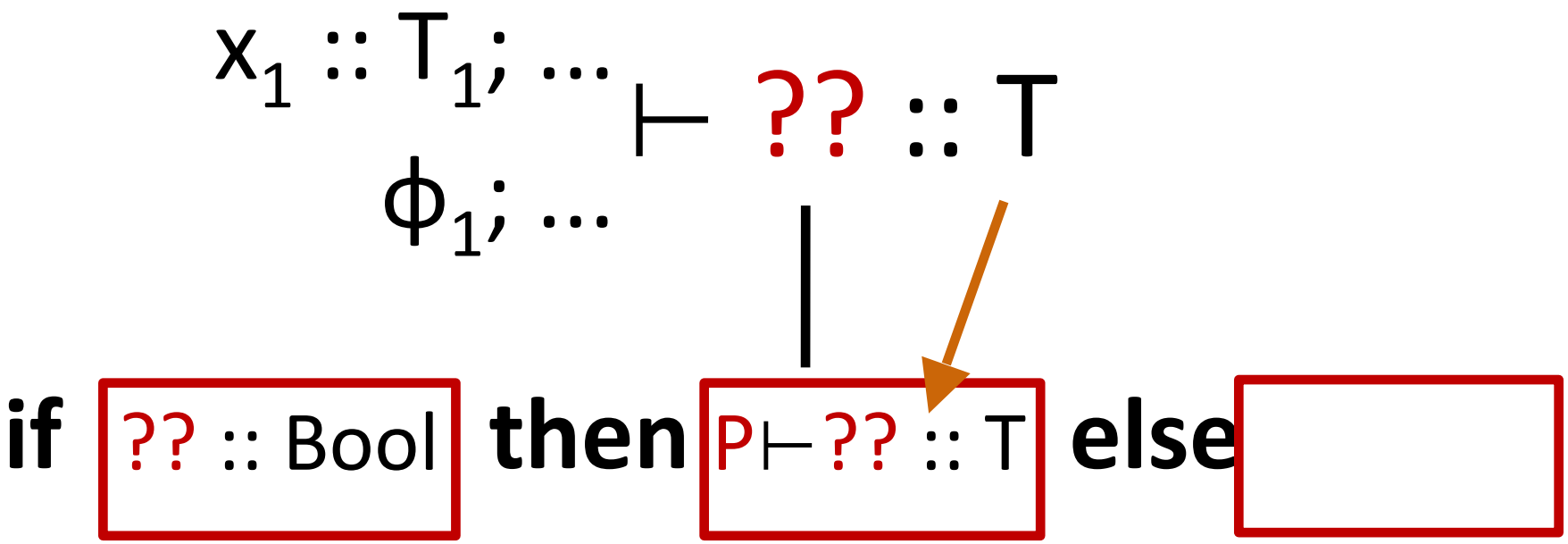
Synthesis from refinement types



I. top-down enumerative search

II. round-trip type checking

Synthesis from refinement types

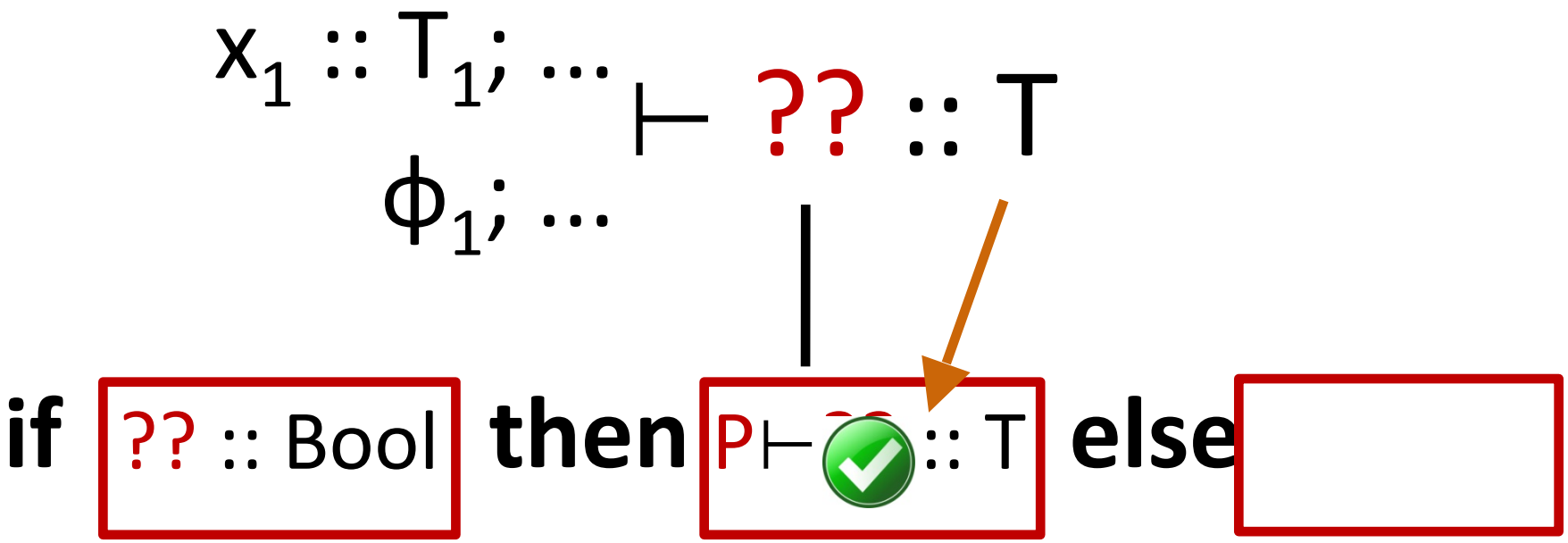


I. top-down enumerative search

II. round-trip type checking

III. condition abduction

Synthesis from refinement types

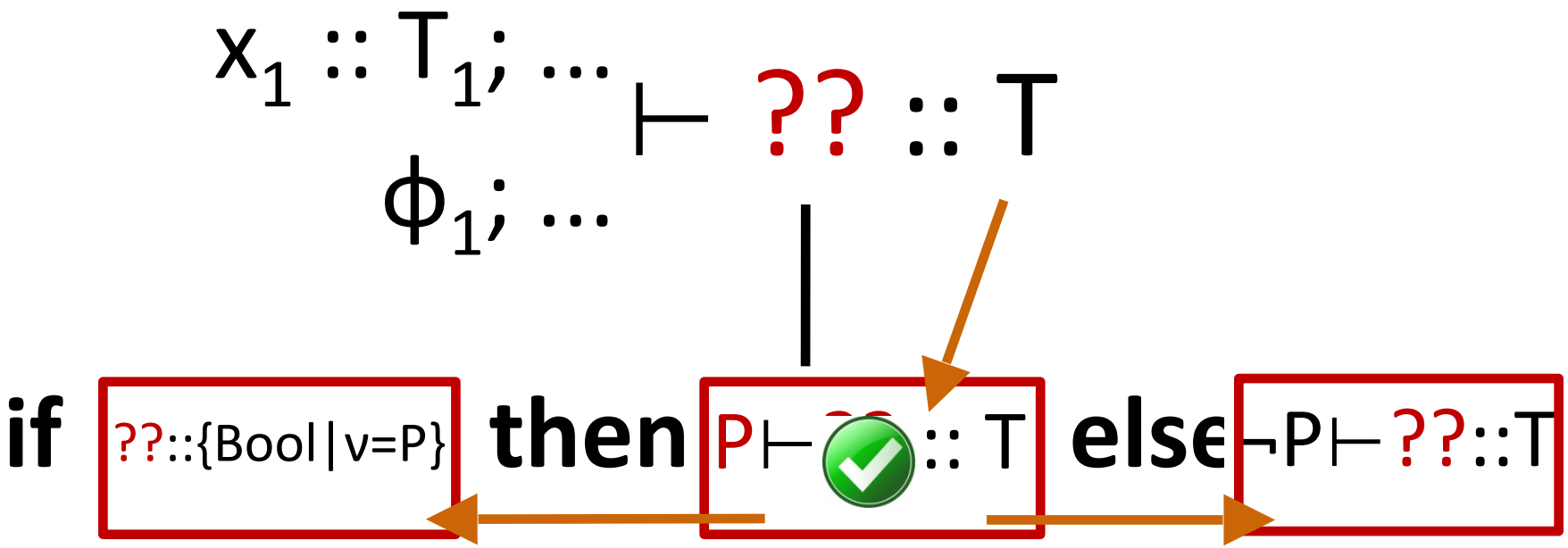


I. top-down enumerative search

II. round-trip type checking

III. condition abduction

Synthesis from refinement types

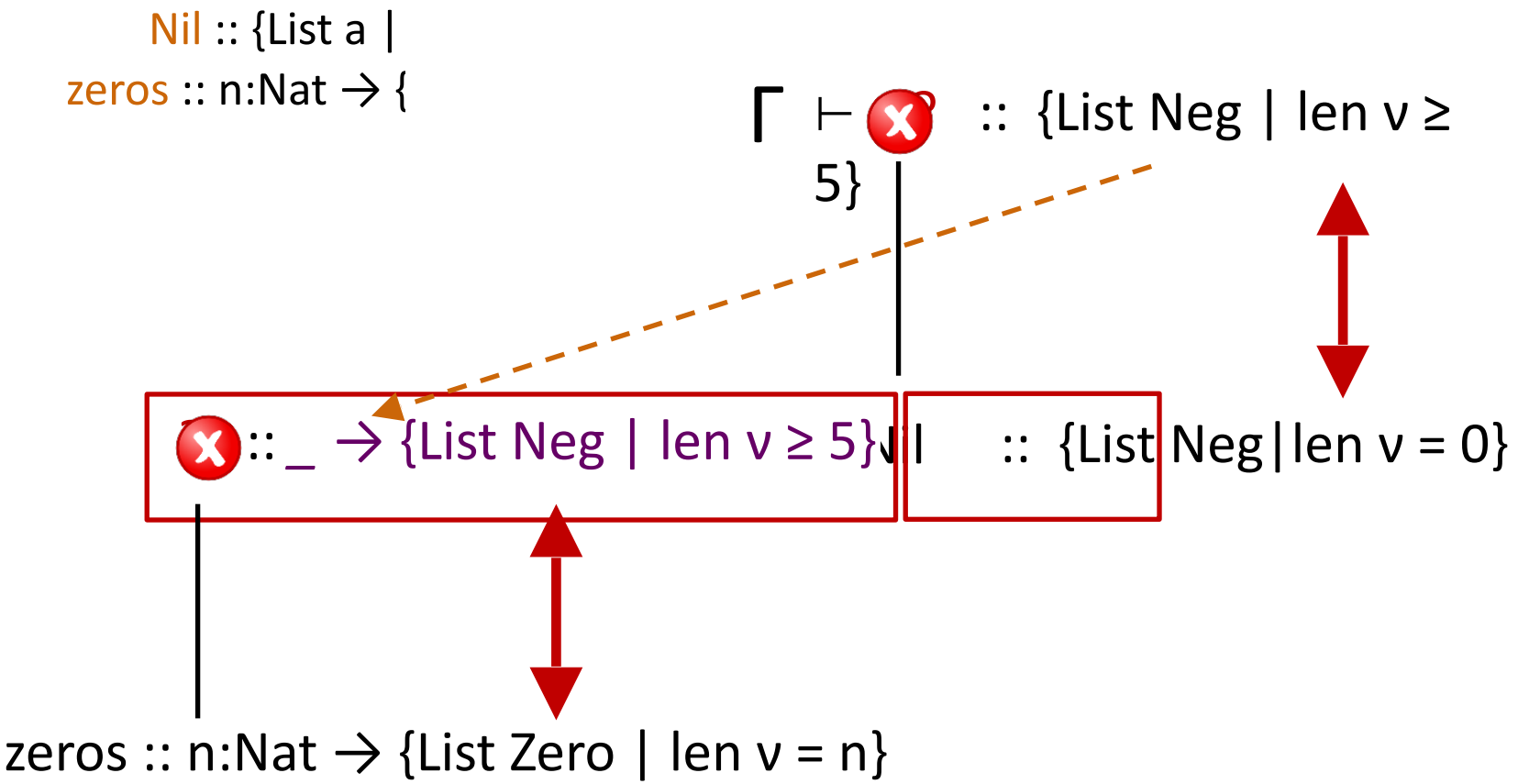


I. top-down enumerative search

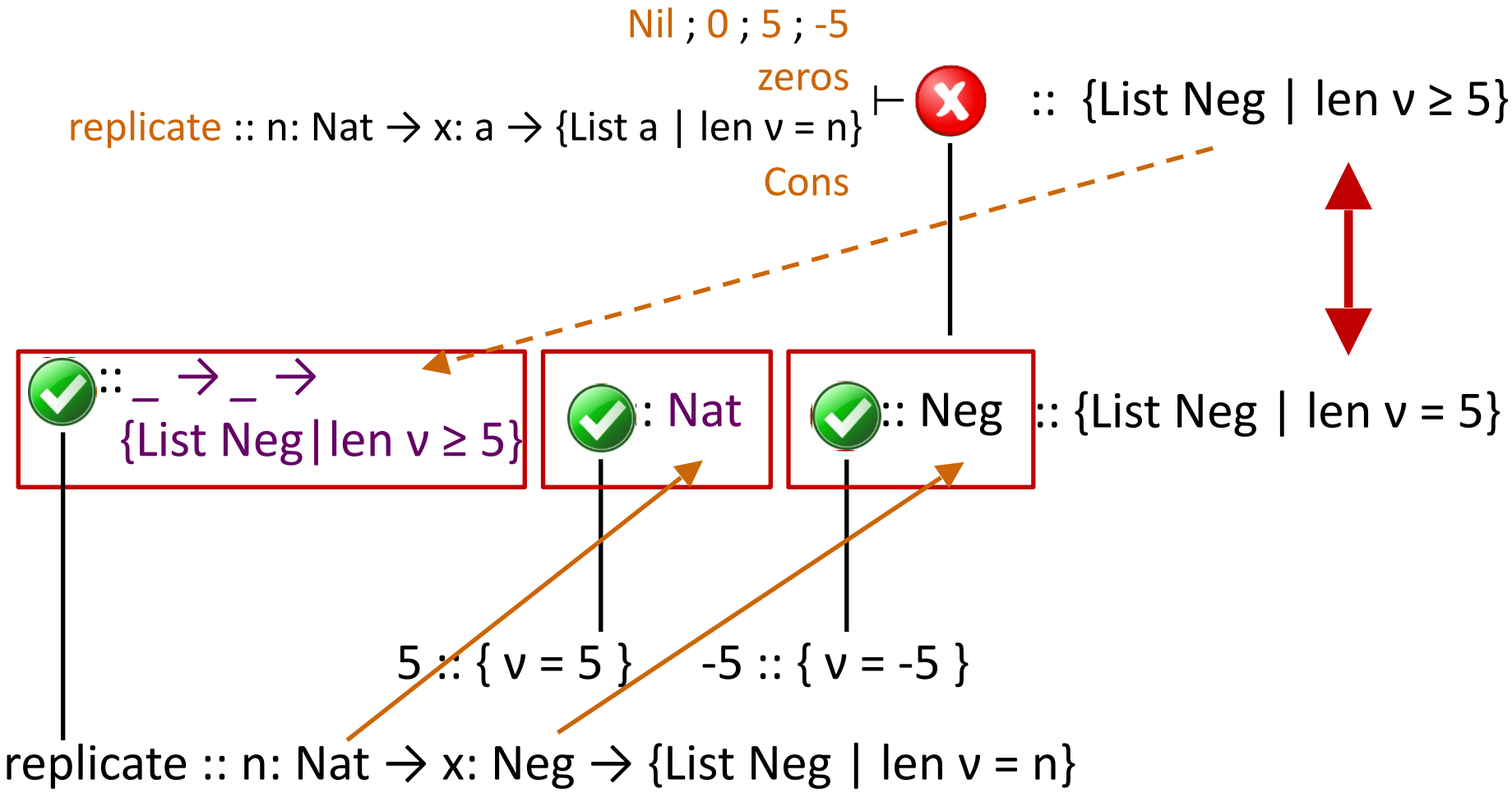
II. round-trip type checking

III. condition abduction

Round-trip type checking



Round-trip type checking



Q3: Can RTTC reject these terms?

`inc ?? :: {Int | v = 5}`

- where `inc :: x:Int → {Int | v = x + 1}`
- NO! don't know if we can find `?? :: {Int | v + 1 = 5}`

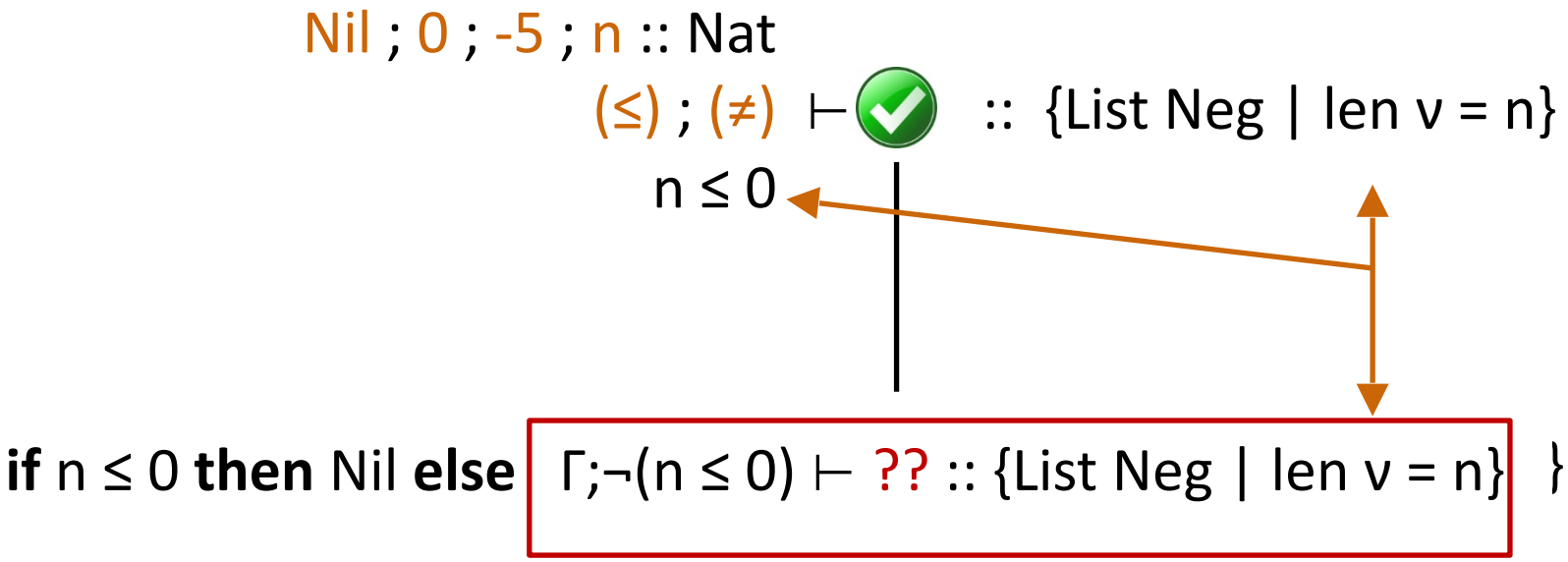
`nats ?? :: List Pos`

- where `nats :: n:Nat → {List Nat| len v = n}`
  `Nat = {Int | v >= 0}, Pos = {Int | v > 0}`
- YES! `n:Nat → {List Nat| len v = n}` not a subtype of
  `_ → List Pos`

`duplicate ?? :: {List Int | len v = 5}`

- where `duplicate :: xs:List a → {List a | len v = 2*(len xs)}`
- YES! using a consistency check (len v = 2*(len xs) ∧ len v = 5 → UNSAT)

Condition abduction



Liquid abduction

